DP & Recursion Architecture

Demystifying when to add +1 (or a value) in the Base Case vs. the Recursive Call

One of the most fundamental concepts in Recursion and Dynamic Programming is knowing exactly where to introduce metrics or state increments. Many students memorize patterns like `return 1 + solve(...);` or `if(base_case) return 1;` without understanding the underlying mechanics. The true answer is elegant: **The location of +1 depends entirely on what your recursive function is designed to return.**

Step 1: Always Define the Meaning of the Function

Before writing a single line of recursive logic, you must write out the state definition in plain language. Define exactly what `f(state) = ?` represents. Everything else follows directly from this definition.

`f(i)` = length of LIS starting from index i

`f(i, target)` = number of subsets that make target

`f(i, target)` = can we make target? (True/False)

The Golden Question: "What am I counting?"

There are only two major paradigms determined by your choice of metric:

Type 1: Counting Progress / Length / Cost

The current state actively contributes to the metric.

Examples: Length of LIS, Height of Tree, Number of Nodes, Minimum Steps, Maximum Sum Path.

`return contribution + solve(...);`

Type 2: Counting Complete Answers

Only a completely built, valid end-solution contributes to the count.

Examples: Count Subsets, Count Paths, Count Partitions, Count Combinations, Count Permutations.

`base case -> return 1;`

Pattern 1: Current State Contributes

Example 1: Linked List Length

Function Meaning: `f(node)` = length of linked list starting from node.

> **Think:** Suppose we evaluate `1 -> 2 -> 3 -> NULL`. At node 1:
> **length = 1 (current node) + length of remaining list**
> Therefore: `return 1 + solve(node->next);`

```
int solve(Node* node){
    if(node == NULL)
        return 0;

    return 1 + solve(node->next);
}
```

Why Base Case Returns 0? Because `NULL` is not a valid node. It contributes absolutely nothing to the length.

Example 2: Height of Binary Tree

Function Meaning: `f(node)` = height of tree rooted at node.

> The current node contributes exactly one level to the vertical structure.
> Therefore: **height = 1 + max(leftHeight, rightHeight)**

```
int height(Node* root){
    if(root == NULL)
        return 0;

    return 1 + max(height(root->left),
                   height(root->right));
}
```

Example 3: Longest Increasing Subsequence (LIS)

Function Meaning: `f(i)` = length of LIS starting from index `i`.

If we decide to select the current element `nums[i]`, that specific element immediately contributes a length of **1** to our subsequence.
Transition: `1 + solve(nextIndex)`

```
int solve(int i){
    int ans = 1;
    for(int j = i + 1; j < n; j++){
        if(nums[j] > nums[i]){
            ans = max(ans, 1 + solve(j));
        }
    }
    return ans;
}
```

***Observation:** Whenever the final answer represents a **Length, Size, Number of Nodes, Number of Levels, or Steps**, you will naturally see the pattern: `return 1 + solve(...);`*

Pattern 2: Completed Solution Contributes

Example 1: Count Subsets with Sum K

Function Meaning: `f(i, target)` = number of valid subsets that can form the target sum.

Suppose `arr = [1, 2]` and `target = 3`.
Path chosen: Take 1 → Take 2. Now, `target == 0`.
A complete, completely valid subset has been fully formed. Thus, this single full path counts as **1 valid sequence**.
Therefore: `return 1;`

```
if(target == 0)
    return 1;
```

Notice we do **not** write `1 + solve(...)` because we are not counting individual elements; we are counting the finalized collection.

Example 2: Count Paths in Grid

Function Meaning: `f(i, j)` = number of unique paths to reach the destination.

When the coordinates match the target boundary `(i, j) == destination`, exactly **one valid path** has successfully completed its journey.
Therefore: `return 1;`

```
if(i == n - 1 && j == m - 1)
    return 1;
```

Example 3: Count Ways to Climb Stairs

Function Meaning: `f(n)` = number of unique ways to reach the nth stair.

When the remaining stairs match `n == 0`, one complete sequential combination of steps has successfully concluded.
Therefore: `return 1;`

```
if(n == 0)
    return 1;
```

Observation: *Whenever the final answer represents a* ***Number of Ways, Number of Paths, Number of Subsets, Combinations, or Permutations****, you will naturally see:* `base case -> return 1;` *because each complete valid terminal state evaluates to 1 completed answer.*

Pattern 3: Current Value Contributes

Sometimes, the contribution is not a uniform step count of `+1`. Instead, the actual value of the current state directly modifies the accumulator.

Example: Maximum Root-to-Leaf Sum

Function Meaning: `f(node)` = maximum node-value sum accumulable from current node to leaf.

The current node contributes its structural value: `node->val`.
Transition: `return node->val + max(left, right);`

```
int solve(Node* root){
    if(root == NULL)
        return 0;

    return root->val + max(solve(root->left),
                           solve(root->right));
}
```

Pattern 4: Boolean Questions

When evaluating existence or reachability, no values or states are being summarized or counted. The question is binary.

Example: Subset Sum

Function Meaning: `f(i, target)` = Can target be successfully formed? (True / False)

> Base Case: `if(target == 0) return true;`
> Why not return 1? Because we are answering a structural status question: **Possible? YES / NO**.

```
if(target == 0)
    return true;
```

The Master Classification Matrix

Before writing down your recursion, utilize this systematic table to identify your target layout:

Category & Key Words	Function Returns	Base Case Value	Problem Examples
Category A: Length / Steps Longest, Shortest, Max Length, Min Steps, Height, Depth, Distance	`int` `return 1 + solve(...)`	`0` length	LIS, Height of Tree, Linked List Length, Minimum Jumps
Category B: Counting Ways Count, Number of Ways, Total Ways, How Many	`int` `base case -> return 1`	`1` (on valid completion)	Count Subsets, Count Partitions, Count Grid Paths, Coin Change II
Category C: Sum / Cost Maximum Sum, Minimum Cost, Profit, Weight	`int` `return value + solve(...)`	`0 profit` / `large value`	Maximum Path Sum, House Robber, Knapsack, Rod Cutting
Category D: Boolean Exists Possible, Exists, Can We, Valid, Reachable	`bool` `return true / false`	`true` / `false`	Subset Sum, Word Break, Graph Reachability, Path Exists

The Ultimate Rule Summary

- **Does the CURRENT state contribute structurally?** (Current node, element, step, level) → **return contribution + solve(...)**
- **Does only a COMPLETE solution path count?** (Valid subset formed, destination reached, valid partition) → **base case -> return 1**
- **Are you evaluating binary validation?** (Nothing is counted, checking raw feasibility) → **return true / false**